



REGLAS Y SCRIPT DE SIMULACIÓN

Aplicación TRIP

Generación de bots, partidas y compras de prueba

PixelPath

www.pixelpath.dev

2026

Tabla de contenido

Tabla de contenido	2
1. Propósito	3
2. Objetivos de la simulación.....	3
3. Archivo y funciones.....	3
Script principal	3
Funciones principales	3
4. Reglas de registros y usuarios	4
Población simulada.....	4
Periodo	4
Crecimiento temporal.....	4
Días y horarios	4
5. Segmentos de jugadores	4
6. Reglas de partidas.....	5
Mapas.....	5
Tipos de muerte	5
7. Reglas de compras	6
8. Integridad y aislamiento.....	6
9. Procedimiento de ejecución	6
Preparación	6
Desde la aplicación	7
Rellenos parciales	7
10. Validaciones posteriores	7
11. Reproducibilidad y limitaciones	8
12. Evidencias de entrega	Error! Bookmark not defined.

1. Propósito

Este documento describe las reglas utilizadas para generar datos simulados del videojuego TRIP. La simulación busca representar el comportamiento de un negocio digital de entretenimiento: registros de jugadores, partidas, progreso, compras y actividad temporal.

El script principal es `supabase_setup_simulacion.sql`

Nota: La simulación es una fuente de datos de prueba. No debe confundirse con registros reales ni ejecutarse sin revisar el entorno de Supabase.

2. Objetivos de la simulación

- Crear una población de jugadores de prueba.
- Generar un historial temporal de registros desde enero de 2024.
- Representar diferentes niveles de participación.
- Generar partidas posteriores al registro de cada jugador.
- Simular abandono progresivo entre mapas.
- Generar compras con una probabilidad controlada.
- Mantener relaciones válidas entre usuarios, partidas, productos y compras.
- Permitir regenerar bots sin eliminar datos de usuarios reales.

3. Archivo y funciones

Script principal

`supabase_setup_simulacion.sql`

Funciones principales

Función	Responsabilidad
<code>admin_regenerate_bots(p_bot_count)</code>	Elimina y crea nuevamente bots de prueba
<code>simulate_player_matches(p_user_id, p_registered_at)</code>	Genera partidas para un jugador
<code>simulate_player_purchases(p_user_id, p_registered_at)</code>	Genera compras para un jugador
<code>admin_run_full_simulation</code>	Coordina la simulación completa
<code>admin_fill_missing_gameplay</code>	Agrega partidas a usuarios sin historial
<code>admin_fill_missing_purchases</code>	Agrega compras a usuarios sin historial

Las tres últimas funciones de administración deben existir previamente en la base de datos y se ejecutan desde el panel /admin/simulacion.

4. Reglas de registros y usuarios

Población simulada

- La cantidad de bots se recibe mediante `p_bot_count`.
- El valor predeterminado es 500.
- La interfaz permite solicitar entre 1 y 2000 bots.
- Los correos se forman como `bot1@gmail.com`, `bot2@gmail.com`, etcétera.
- Los nombres de usuario se forman como `bot_1`, `bot_2`, etcétera.
- Cada bot recibe un UUID generado con `gen_random_uuid()`.
- Los bots se crean con rol `user`.
- Las contraseñas simuladas se generan con el valor utilizado por la función SQL.

Periodo

- Fecha inicial: `2024-01-01`.
- Fecha final: aproximadamente 14 días antes de la fecha actual.
- No se generan registros de bots posteriores a la fecha final calculada.
- El número de meses se calcula dinámicamente entre ambas fechas.

Crecimiento temporal

La cantidad esperada de registros por mes utiliza una curva creciente:

```
peso mensual = potencia del número de mes x factor aleatorio
```

El factor aleatorio evita que el crecimiento sea perfectamente lineal. La simulación representa un lanzamiento pequeño que incrementa su audiencia con el tiempo, pero conserva meses con variaciones.

Días y horarios

- Viernes, sábado y domingo tienen mayor probabilidad de registro.
- Los demás días se reducen mediante un factor de probabilidad.
- Los horarios se seleccionan principalmente entre las 13:00 y las 23:00.
- También pueden aparecer registros entre las 00:00 y la 01:00, y entre las 09:00 y las 11:00.
- Los minutos se generan dentro de la hora seleccionada.

5. Segmentos de jugadores

Cada jugador recibe un nivel de participación mediante un valor aleatorio `v_tier`.

Segmento	Proporción aprox.	Partidas esperadas	Comportamiento
Prueba y abandono	45%	1 a 3	Juega durante los primeros días y normalmente no continúa
Casual	35%	5 a 18	Regresa ocasionalmente durante varias semanas
Regular	15%	18 a 45	Juega durante periodos más largos
Fan	5%	45 a 90	Tiene actividad frecuente y prolongada

Estas proporciones son reglas de simulación, no resultados observados de usuarios reales.

6. Reglas de partidas

Cada partida se genera con las siguientes reglas:

- La primera partida ocurre entre 0 y 180 minutos después del registro.
- Las siguientes partidas se separan por intervalos de días según el segmento.
- Si una partida quedaría después de la fecha actual, se detiene la generación.
- La partida siempre pertenece a un user_id existente.
- La fecha de partida nunca debe ser anterior al registro del jugador.
- La duración se genera entre 60 y 599 segundos.
- Aproximadamente 55% de las partidas tienen una muerte registrada.
- El número de consumos se genera entre 0 y 5.
- El daño recibido se genera entre 0 y 99.
- Si el jugador muere, la vida final se genera entre 0 y 29.
- Si no muere, la vida final se genera entre 40 y 100.

Mapas

Los mapas se recorren mediante una regla de abandono progresivo:

Calle Vacía -> Habitación -> Hospital -> Callejón -> Refugio -> Recuerdo

La generación inicia en el primer mapa. Cada avance al siguiente mapa depende de una probabilidad aproximada de 62%, por lo que los mapas posteriores reciben menos actividad.

Tipos de muerte

Los valores posibles son:

- enemigos.
- sobredosis.
- abstinencia.

- colapso.
- caídas.

7. Reglas de compras

- Aproximadamente 28% de los jugadores realiza al menos una compra.
- Los jugadores que compran suelen realizar una o dos compras.
- Una proporción pequeña puede realizar entre tres y cinco compras.
- La fecha de compra se genera después del registro.
- Las compras no deben quedar en el futuro.
- El producto se selecciona del catálogo existente.
- El precio pagado se toma del precio del producto al momento de generar la compra.
- Si no existen productos, no se genera la compra.
- La compra se relaciona con user_id y product_id.

La función de la tienda maneja como repetibles los productos de tipo `mesh`; los tipos `juego`, `asset` y `otro` se tratan como compra única desde la interfaz.

8. Integridad y aislamiento

La simulación debe cumplir estas reglas:

1. Cada bot debe tener un perfil válido.
2. Cada partida debe pertenecer a un jugador existente.
3. Cada compra debe pertenecer a un jugador y a un producto existente.
4. Ninguna partida debe ocurrir antes del registro del jugador.
5. Ninguna compra debe ocurrir antes del registro del jugador.
6. No deben generarse partidas ni compras futuras.
7. Los administradores no deben contarse como jugadores en los KPI generales.
8. La regeneración debe identificar bots por el patrón bot%`@gmail.com`.
9. La eliminación de bots no debe afectar usuarios reales.
10. La ejecución debe realizarse con una cuenta que tenga rol admin.

9. Procedimiento de ejecución

Preparación

11. Confirma que trabajas en un proyecto de Supabase de desarrollo o prueba.
12. Verifica que existan las tablas `profiles`, `products`, `purchases`, `game_matches` y `progress`.
13. Verifica que existan las funciones administrativas requeridas.

14. Confirma que la cuenta actual tenga rol admin.
15. Revisa que existan productos antes de generar compras.
16. Ejecuta el archivo desde el SQL Editor de Supabase.

Desde la aplicación

17. Inicia sesión como administrador.
18. Abre /admin/simulacion.
19. Introduce la cantidad de bots.
20. Selecciona Regenerar todo para crear bots, partidas y compras.
21. Espera el mensaje de resultado.
22. Abre /dashboard_admin.
23. Revisa usuarios, ventas, ingresos, retención y partidas.
24. Usa los rellenos parciales solo cuando sea necesario.

Rellenos parciales

Rellenar partidas faltantes: completa el historial de usuarios sin partidas.

Rellenar compras faltantes: completa el historial de usuarios sin compras.

Nota: Estas opciones no deben utilizarse como sustituto de una validación de calidad.

10. Validaciones posteriores

Después de ejecutar la simulación, registra al menos:

Validación	Resultado esperado	Resultado obtenido
Cantidad de bots	Igual a p_bot_count	1000
Perfiles de bots	Todos tienen role = user	Correcto
Partidas sin usuario	0	0
Compras sin usuario	0	0
Compras sin producto	0	0
Partidas anteriores al registro	0	0
Compras anteriores al registro	0	0
Fechas futuras	0	0
Duraciones negativas	0	0
Productos disponibles	Mayor que 0	17

11. Reproducibilidad y limitaciones

La simulación actual utiliza `random()` de PostgreSQL y no configura una semilla. Por ese motivo, dos ejecuciones pueden producir valores diferentes aunque se utilice la misma cantidad de bots.

Para mejorar la reproducibilidad académica se recomienda:

- implementar una semilla controlada;
- registrar la fecha y hora de ejecución;
- registrar el número de bots;
- conservar una copia del dataset original;
- conservar una copia del dataset procesado;
- documentar la versión del script;
- guardar los resultados de las validaciones.